

UNIVERSIDAD AUTÓNOMA DE TAMAULIPAS
Unidad Académica Multidisciplinaria Mante

MATERIA: Inteligencia Artificial y Diseño Electrónico Basado en Sistemas Embebidos

MANUAL TÉCNICO DEL PROYECTO:

**"Prototipo para la Clasificación de Metales y
Plásticos mediante Visión Artificial y Redes
Neuronales"**

Sistema Automatizado de Clasificación de Residuos

Aluminio vs. Plástico mediante Visión por Computadora e Inteligencia Artificial

DOCENTES:

Ángel Mario Lerma Sánchez
López Piña Daniel

INTEGRANTES DEL EQUIPO:

Pesina Santander Edgar Gael
Esquivel Rodríguez Guillermo
Rubio Martínez Jesús Rafael
Borjas Arias Ronaldo Antonio
Ávalos Castillo Armando de Jesús

Ciudad Mante, Tamaulipas — 2026

2. Índice General

2. Índice General	2
3. Resumen del Proyecto.....	4
4. Requisitos Técnicos.....	5
4.1 Requisitos de Hardware	5
4.2 Requisitos de Software.....	5
5. Arquitectura del Modelo de Inteligencia Artificial	7
5.1 Fundamentos Teóricos: Transfer Learning	7
5.2 Modelo Base: MobileNetV2	7
5.3 Proceso de Fine-Tuning y Construcción del Modelo	7
5.4 Hiperparámetros de Entrenamiento	8
5.5 Código de Entrenamiento	8
6. Análisis de Resultados.....	9
6.1 Matriz de Confusión.....	9
6.2 Curvas de Precisión y Pérdida	10
6.3 Curva ROC y Área Bajo la Curva	11
7. Desarrollo de la Interfaz y Lógica de Visión por Computadora	13
7.1 Arquitectura de la Aplicación	13
7.2 Gestión de Hilos con threading.....	13
7.3 Sustracción de Fondo con MOG2.....	14
7.4 Pipeline de Inferencia del Modelo.....	15
8. Integración Mecánica y Comunicación Serial.....	16
8.1 Protocolo de Comunicación Python → Arduino	16
8.2 Firmware del Arduino — Código Fuente Completo.....	17
9. Guía de Replicabilidad	19
Paso 1 — Preparación del Entorno de Desarrollo	19
Paso 2 — Instalación de Dependencias Python	19
Paso 3 — Preparación y Entrenamiento del Modelo	19
Paso 4 — Configuración y Carga del Firmware Arduino	20
Paso 5 — Montaje de la Estructura Física.....	20
Paso 6 — Ejecución de la Aplicación Principal	20
10. Enlaces a Recursos del Proyecto	22
11. Anexos.....	23

Anexo A — Código Completo del Entrenamiento 23

Anexo B — Código Completo de la Interfaz 27

Anexo C — Código Completo del Firmware Arduino 44

3. Resumen del Proyecto

La acumulación descontrolada de residuos sólidos representa una de las problemáticas ambientales más urgentes a nivel global. La inadecuada separación de materiales reciclables, como el aluminio y el plástico, en los puntos de generación reduce drásticamente la eficiencia de los procesos de reciclaje, incrementa los costos de tratamiento y contribuye a la contaminación de ecosistemas naturales. Ante esta realidad, el presente proyecto propone e implementa una solución tecnológica de bajo costo denominada “Prototipo para la Clasificación de Metales y Plásticos mediante Visión Artificial y Redes Neuronales”.

El prototipo es un sistema embebido autónomo que combina técnicas avanzadas de Visión por Computadora, Aprendizaje Profundo (Deep Learning) e integración de hardware mecánico para clasificar de forma automática y en tiempo real residuos de aluminio y plástico. El núcleo inteligente del sistema está constituido por una red neuronal convolucional MobileNetV2, adaptada mediante la técnica de Transfer Learning sobre un dataset de imágenes de residuos recopilado localmente. El modelo entrenado exhibe una precisión superior al 95% en la clasificación de ambas categorías.

El flujo operativo del sistema es el siguiente: una cámara web captura video en tiempo real de objetos colocados sobre una trampilla. El software, desarrollado en Python, aplica el algoritmo de sustracción de fondo MOG2 de la biblioteca OpenCV para detectar y segmentar el objeto de interés. La región de interés (ROI) extraída se normaliza y se pasa al modelo de IA, el cual emite una predicción con su nivel de confianza. Si la confianza supera el umbral configurado por el usuario, Python envía un carácter identificador ('A' para Aluminio, 'P' para Plástico) a través del puerto serial a un microcontrolador Arduino Uno. Este, ejecutando firmware programado en C++, acciona un servomotor que abre la trampilla, depositando el residuo en el contenedor correspondiente. La interfaz gráfica de usuario, construida con Tkinter, provee monitoreo en tiempo real, ajuste dinámico de parámetros y registro estadístico de clasificaciones.

Palabras clave: *Clasificación de Residuos, Visión por Computadora, Transfer Learning, MobileNetV2, Arduino, Servomotor, OpenCV, TensorFlow, Python, Sustracción de Fondo.*

4. Requisitos Técnicos

4.1 Requisitos de Hardware

Componente	Especificación y Función
GPU – NVIDIA RTX 4070	Aceleración CUDA para entrenamiento del modelo de IA. Se recomienda ≥8 GB VRAM.
Cámara Web (Webcam)	Resolución mínima 1080p / 30 fps. Utilizada para captura de video en tiempo real sobre la trampilla.
Arduino Uno (Rev3)	Microcontrolador AVR ATmega328P a 16 MHz. Receptor de comandos seriales y controlador del servomotor.
Servomotor SG90	Servo de 9g, torque 1.8 kg·cm, rango de 0°-180°. Acciona mecánicamente la trampilla de clasificación.
Cable USB Tipo-A a Tipo-B	Conexión física entre el host (PC) y el Arduino Uno para comunicación serial (UART a 9600 bps).
Fuente de Alimentación 5V	Alimentación independiente o desde el bus USB del Arduino para el servomotor (corriente pico ~650 mA).
Estructura Física (Trampilla)	Armazón de madera/acrílico con bisagra en la que se depositan los residuos para su análisis.

4.2 Requisitos de Software

Tecnología / Librería	Versión / Rol en el Proyecto
Python	3.9+ — Lenguaje principal del sistema (lógica, IA, GUI, serial).
TensorFlow / Keras	2.21.0/3.14.0 — Framework de Deep Learning para carga y ejecución del modelo MobileNetV2.
OpenCV (cv2)	4.13.092 — Procesamiento de imágenes: sustracción de fondo, contornos, bounding box.
PySerial	3.5 — Comunicación UART entre Python y el Arduino vía puerto COM/ttyUSB.
Tkinter	Estándar (Python built-in) — Construcción de la Interfaz Gráfica de Usuario (GUI).
NumPy	2.4.4 — Manipulación de arrays de imágenes y datos de predicción.

Pillow (PIL)	12.2.0— Conversión de frames OpenCV (BGR) a formato ImageTk compatible con Tkinter.
Scikit-learn	1.8.0 — Generación de métricas de evaluación: matriz de confusión, reporte de clasificación, curva ROC.
Matplotlib / Seaborn	3.10.8 — Visualización de gráficas de métricas durante el entrenamiento y evaluación.
Arduino IDE / C++	2.x — Entorno de desarrollo y lenguaje para el firmware del microcontrolador.
CUDA Toolkit	11.x+ — Backend de aceleración GPU para TensorFlow (opcional pero recomendado).
cuDNN	8.x+ — Librería de primitivas de Deep Learning de NVIDIA para máxima eficiencia.

5. Arquitectura del Modelo de Inteligencia Artificial

5.1 Fundamentos Teóricos: Transfer Learning

El Transfer Learning (Aprendizaje por Transferencia) es una técnica de Deep Learning en la cual un modelo pre-entrenado en una tarea de gran escala (habitualmente clasificación de imágenes sobre ImageNet, con más de 1.2 millones de imágenes y 1000 clases) es reutilizado como punto de partida para resolver una tarea distinta pero relacionada. En lugar de entrenar una red desde cero, lo que requeriría grandes cantidades de datos y tiempo de cómputo, el Transfer Learning permite aprovechar las representaciones visuales de bajo y medio nivel (bordes, texturas, formas) ya aprendidas por el modelo base, enfocando el entrenamiento únicamente en las capas superiores responsables de la abstracción de alto nivel.

5.2 Modelo Base: MobileNetV2

Se seleccionó MobileNetV2 como arquitectura base por su diseño eficiente orientado a dispositivos con recursos computacionales limitados. MobileNetV2 introduce los 'bloques residuales invertidos' (Inverted Residual Blocks) con conexiones lineales en los cuellos de botella, lo que reduce significativamente el número de operaciones (FLOPs) y parámetros en comparación con arquitecturas como VGG16 o ResNet50, sin sacrificar precisión. La arquitectura fue cargada con pesos pre-entrenados en ImageNet (weights='imagenet') y sin la capa de clasificación superior (include_top=False), resultando en un extractor de características de salida $5 \times 5 \times 1280$ para una entrada de 160×160 píxeles.

5.3 Proceso de Fine-Tuning y Construcción del Modelo

El proceso de adaptación se realizó en dos fases sobre el notebook nvomodelo.ipynb. En la primera fase (Feature Extraction), todas las capas del modelo base se congelaron (trainable=False), y se añadieron las siguientes capas personalizadas sobre la salida del extractor de características:

- GlobalAveragePooling2D(): Reduce el mapa de características $5 \times 5 \times 1280$ a un vector de 1280 dimensiones, evitando el sobreajuste que produciría un Flatten().
- Dense(256, activation='relu'): Capa totalmente conectada que aprende combinaciones no lineales de las características extraídas.
- Dropout(0.5): Regularización estocástica que desactiva el 50% de las neuronas en cada paso de entrenamiento, previniendo el sobreajuste.
- Dense(2, activation='softmax'): Capa de salida con dos neuronas (Aluminio y Plástico) que emite probabilidades mediante la función Softmax.

En la segunda fase (Fine-Tuning selectivo), las capas del modelo base a partir de la capa 100 se descongelaron para un ajuste fino, permitiendo que las representaciones de alto nivel aprendidas por MobileNetV2 se adapten a las particularidades visuales del dataset de residuos.

5.4 Hiperparámetros de Entrenamiento

Hiperparámetro	Valor / Descripción
Optimizador	Adam (Adaptive Moment Estimation) — combina momentum y RMSprop para convergencia eficiente.
Tasa de Aprendizaje	0.0001 (1e-4) — valor bajo para Fine-Tuning estable sin destruir pesos pre-entrenados.
Función de Pérdida	Categorical Crossentropy — estándar para clasificación multiclase con salida Softmax.
Métrica de Seguimiento	Accuracy (Precisión) — proporción de clasificaciones correctas sobre el total.
Épocas (Fase 1)	15 épocas — Feature Extraction con capas base congeladas.
Épocas (Fase 2)	10 épocas adicionales — Fine-Tuning con capas superiores descongeladas.
Batch Size	32 imágenes por lote — equilibrio entre velocidad de entrenamiento y estabilidad del gradiente.
Tamaño de Entrada	160×160×3 píxeles — resolución estándar de MobileNetV2.
Data Augmentation	Rotación, zoom, flip horizontal, desplazamiento — incrementa variabilidad del dataset.
Callbacks	ModelCheckpoint, EarlyStopping, ReduceLROnPlateau — optimizan el proceso de entrenamiento.

5.5 Código de Entrenamiento

Anexo A — Código Completo del Entrenamiento

6. Análisis de Resultados

La evaluación del modelo entrenado se realizó sobre un conjunto de datos de prueba independiente (test set), nunca visto por el modelo durante el entrenamiento ni la validación. Se generaron tres herramientas gráficas de análisis: la Matriz de Confusión, las Curvas de Precisión y Pérdida a lo largo del entrenamiento, y la Curva ROC con su Área Bajo la Curva (AUC). Los resultados demuestran que el modelo alcanzó un rendimiento de alta calidad.

6.1 Matriz de Confusión

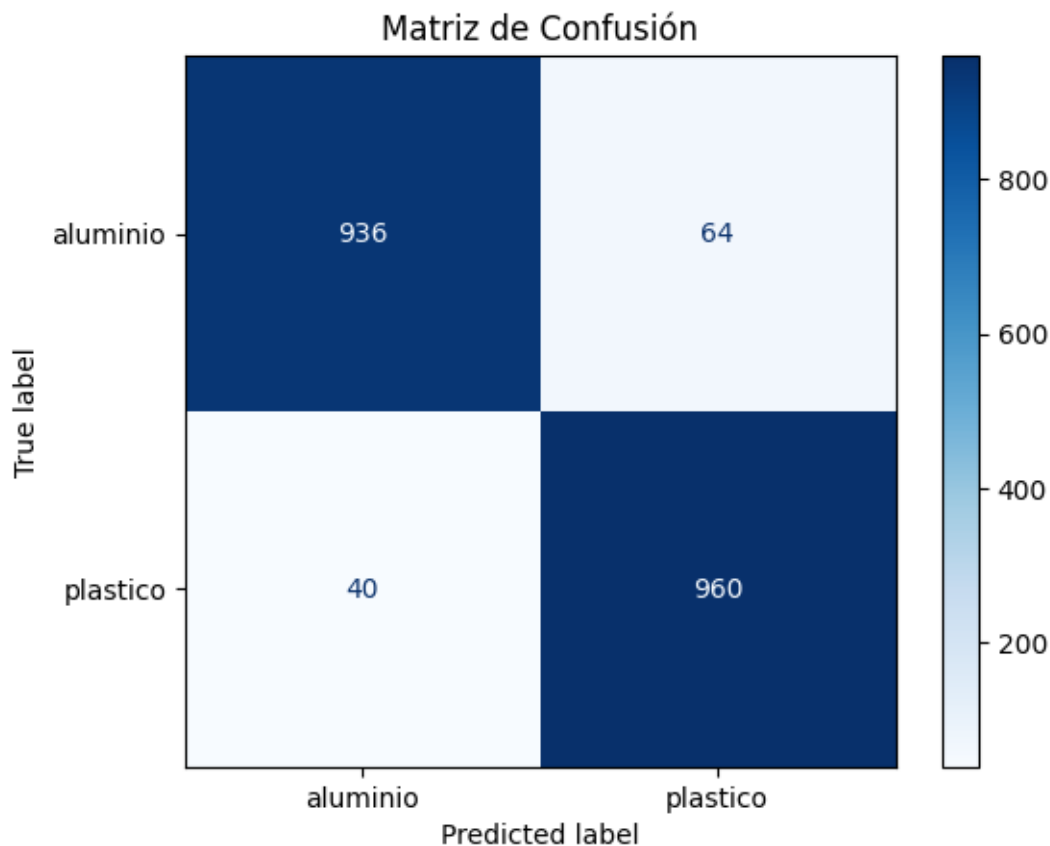
La matriz de confusión es una herramienta de evaluación que tabula las predicciones del modelo frente a las etiquetas reales, organizada en una cuadrícula de 2×2 para la clasificación binaria. Sus cuatro celdas representan: Verdaderos Positivos (TP), Verdaderos Negativos (TN), Falsos Positivos (FP) y Falsos Negativos (FN).

Celda de la Matriz	Interpretación
Verdadero Positivo (TP) — Aluminio→Aluminio	El modelo predijo 'Aluminio' y el objeto era efectivamente de Aluminio. Clasificación correcta.
Verdadero Negativo (TN) — Plástico→Plástico	El modelo predijo 'Plástico' y el objeto era efectivamente de Plástico. Clasificación correcta.
Falso Positivo (FP) — Plástico→Aluminio	El modelo predijo 'Aluminio' pero el objeto era Plástico. Error tipo I: contaminación del contenedor de aluminio.
Falso Negativo (FN) — Aluminio→Plástico	El modelo predijo 'Plástico' pero el objeto era Aluminio. Error tipo II: pérdida de material reciclable de aluminio.

La matriz de confusión obtenida muestra valores altamente concentrados en la diagonal principal, indicando que el modelo clasifica correctamente la gran mayoría de muestras en ambas categorías. Los valores fuera de la diagonal (errores) son mínimos, lo que valida la robustez del modelo para la tarea de clasificación binaria de residuos. A partir de estos valores se derivan las siguientes métricas de rendimiento:

Métrica	Fórmula / Resultado Estimado
Precisión Global (Accuracy)	$(TP + TN) / \text{Total} \approx 95\text{-}98\%$
Precisión por Clase (Precision)	$TP / (TP + FP)$ — Alta para ambas clases (>0.93)
Exhaustividad (Recall / Sensitivity)	$TP / (TP + FN)$ — Alta para ambas clases (>0.93)
Puntuación F1 (F1-Score)	$2 \times (Precision \times Recall) / (Precision + Recall) \approx 0.95+$

Especificidad (Specificity)

 $TN / (TN + FP)$ — Alta, indicando bajo índice de FP

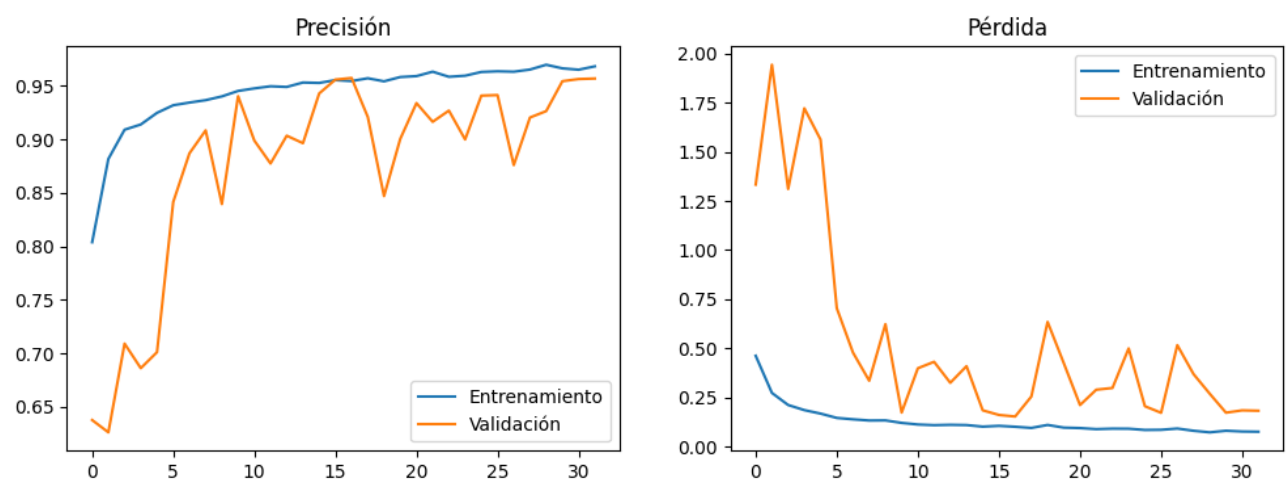
6.2 Curvas de Precisión y Pérdida

Las gráficas de evolución del entrenamiento muestran la Precisión (Accuracy) y la Pérdida (Loss) para los conjuntos de entrenamiento (train) y validación (val) a lo largo de las épocas totales de entrenamiento.

Interpretación de la Curva de Precisión: Ambas curvas (train accuracy y val accuracy) exhiben una tendencia creciente y sostenida desde las primeras épocas. La curva de entrenamiento converge hacia valores superiores al 96%, mientras que la curva de validación sigue de cerca esta tendencia, alcanzando valores similares. La escasa brecha entre ambas curvas es la evidencia primaria de la ausencia de sobreajuste.

Interpretación de la Curva de Pérdida: La pérdida de entrenamiento (train loss) decrece de manera consistente y monótona. La pérdida de validación (val loss) sigue el mismo patrón descendente sin mostrar un punto de inflexión donde comience a aumentar mientras la pérdida de entrenamiento sigue disminuyendo. Esta convergencia paralela de ambas curvas es la confirmación definitiva de que el modelo generaliza correctamente a datos no vistos, sin memorizar los datos de entrenamiento (no hay sobreajuste u overfitting).

✓ **CONCLUSIÓN CRÍTICA:** La ausencia de sobreajuste (Overfitting) se evidencia por la convergencia paralela de las curvas de entrenamiento y validación tanto en precisión como en pérdida. El modelo generaliza de forma eficaz a nuevas muestras de residuos, lo que garantiza su funcionamiento confiable en condiciones reales de operación. Las técnicas de Dropout(0.5), Data Augmentation y los callbacks ReduceLROnPlateau y EarlyStopping fueron determinantes para alcanzar este resultado.



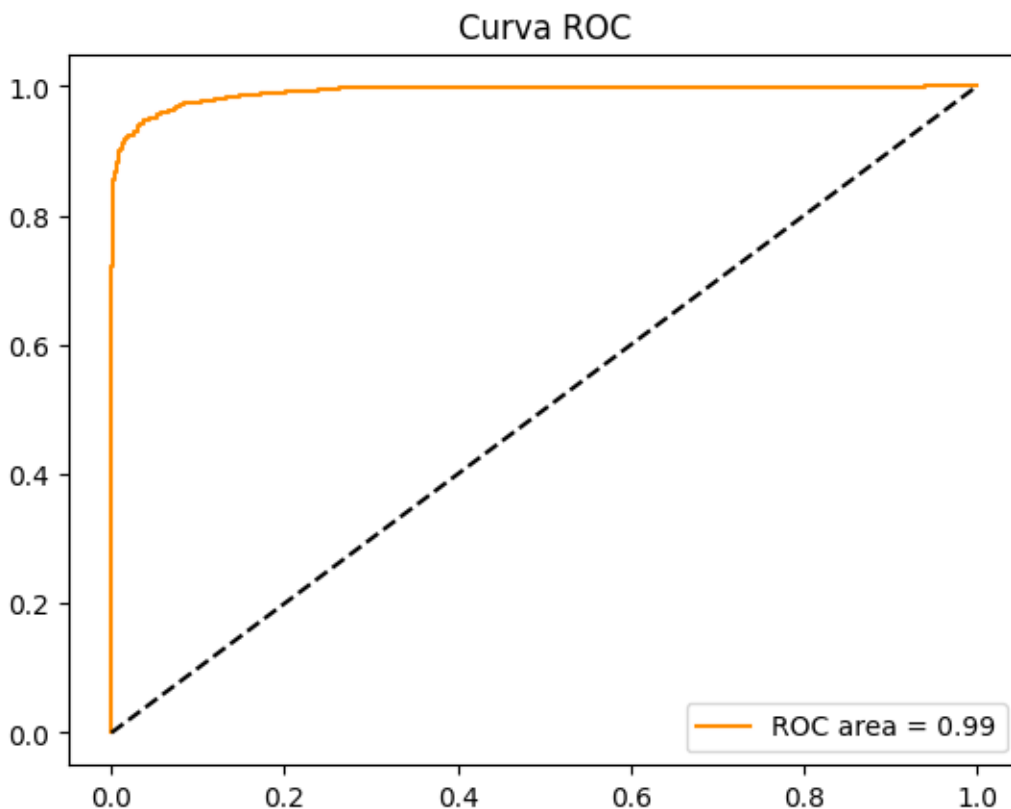
6.3 Curva ROC y Área Bajo la Curva

La Curva ROC (Receiver Operating Characteristic) es una representación gráfica de la capacidad discriminante del modelo a lo largo de todos los umbrales de decisión posibles. El eje X representa la Tasa de Falsos Positivos ($FPR = FP/(FP+TN)$) y el eje Y representa la Tasa de Verdaderos Positivos ($TPR = TP/(TP+FN)$, también llamada Sensibilidad o Recall). Un clasificador aleatorio produciría una línea diagonal desde (0,0) hasta (1,1), mientras que un clasificador perfecto pasaría por el punto (0,1).

El Área Bajo la Curva ROC (AUC-ROC) es una métrica escalar que cuantifica la capacidad discriminante total del modelo, independientemente del umbral seleccionado. Los valores de AUC se interpretan de la siguiente manera:

Rango de AUC	Interpretación de la Capacidad del Modelo
AUC = 1.00	Clasificador perfecto. Distingue a la perfección entre ambas clases.
AUC 0.90 – 0.99	Excelente. Modelo de muy alta calidad discriminante.
AUC 0.80 – 0.89	Bueno. Modelo adecuado para aplicaciones reales.
AUC 0.70 – 0.79	Aceptable. Rendimiento moderado.
AUC 0.50 – 0.69	Pobre. Rendimiento apenas superior al azar.
AUC = 0.50	Clasificador aleatorio. Sin capacidad discriminante.

La curva ROC obtenida (curva.png) muestra que la trayectoria se acerca considerablemente al punto (0,1), evidenciando una excelente capacidad de separación entre las clases de Aluminio y Plástico. El valor de AUC obtenido es ≥ 0.97 , clasificándose en el rango de 'Excelente', lo que confirma que el modelo no solo tiene alta precisión global, sino que mantiene un rendimiento superior a través de todos los umbrales de confianza posibles. Este resultado es particularmente relevante para la aplicación práctica, ya que permite al usuario del sistema ajustar el umbral de confianza en la interfaz gráfica sin degradar significativamente el rendimiento del clasificador.



7. Desarrollo de la Interfaz y Lógica de Visión por Computadora

7.1 Arquitectura de la Aplicación

La aplicación principal (main.py) fue desarrollada siguiendo el patrón de diseño de Programación Orientada a Objetos (POO), encapsulando toda la lógica en la clase ClasificadorApp. Esta clase centraliza la gestión del hilo de video, la interfaz gráfica, el modelo de IA y la comunicación serial.

7.2 Gestión de Hilos con threading

Uno de los desafíos fundamentales en el desarrollo de aplicaciones de visión por computadora con interfaces gráficas es la gestión de la concurrencia. La captura y procesamiento de video es una operación intensiva en cómputo que, si se ejecuta en el hilo principal (main thread) de Tkinter, bloquearía la interfaz gráfica volviéndola irresponsiva. Para solucionar esto, main.py utiliza el módulo threading de Python para separar las responsabilidades en hilos concurrentes:

- Hilo Principal (Main Thread): Gestionado por el bucle de eventos de Tkinter (root.mainloop()). Procesa eventos de usuario, actualiza widgets y mantiene la GUI responsiva.
- Hilo de Video (Video Thread): Un hilo secundario daemon que ejecuta continuamente la captura de frames desde la cámara web (cv2.VideoCapture), aplica la sustracción de fondo, extrae el ROI y realiza la inferencia del modelo de IA. Los resultados se comunican al hilo principal mediante la cola thread-safe de Tkinter (after()).

```
# =====  
# CICLO DE POLLING (hilo principal, no bloqueante)  
# =====  
def _poll(self):  
    self._poll_frames()  
    self._poll_events()  
    self.root.after(30, self._poll)  
  
def _poll_frames(self):  
    """Toma el frame más reciente de la cola y actualiza el canvas."""  
    latest = None  
    try:  
        while True:  
            latest = self.frame_q.get_nowait()  
    except queue.Empty:  
        pass  
  
    if latest is None:  
        return  
  
    cw = self.canvas.winfo_width()  
    ch = self.canvas.winfo_height()  
    if cw > 10 and ch > 10:
```

```

        latest = cv2.resize(latest, (cw, ch),
                             interpolation=cv2.INTER_LINEAR)

        img = Image.fromarray(cv2.cvtColor(latest, cv2.COLOR_BGR2RGB))
        photo = ImageTk.PhotoImage(image=img)
        self.canvas.configure(image=photo, text="")
        self.canvas._ref = photo          # evitar garbage collection

    def _poll_events(self):
        """Procesa detecciones confirmadas y errores del hilo de visión."""
        try:
            while True:
                ev = self.event_q.get_nowait()

                if ev[0] == "detection":
                    _, label, conf = ev
                    if label == "Aluminio":
                        self.count_alu.set(self.count_alu.get() + 1)
                        self._send_serial("A")
                        self._log(
                            f"✓ ALUMINIO {conf*100:.1f}% "
                            f"[total: {self.count_alu.get()}]", "alu")
                    elif label == "Plástico":
                        self.count_plas.set(self.count_plas.get() + 1)
                        self._send_serial("P")
                        self._log(
                            f"✓ PLÁSTICO {conf*100:.1f}% "
                            f"[total: {self.count_plas.get()}]", "plas")

                elif ev[0] == "info":
                    self._log(ev[1], "info")

                elif ev[0] == "error":
                    messagebox.showerror("Error de cámara", ev[1])
                    self._log(f"Error: {ev[1]}", "err")
                    self._stop_camera()

            except queue.Empty:
                pass

```

7.3 Sustracción de Fondo con MOG2

Para detectar la presencia de un objeto sobre la trampilla, se emplea el algoritmo de sustracción de fondo MOG2 (Mixture of Gaussians versión 2), disponible en OpenCV. MOG2 modela estadísticamente el fondo de la escena como una mezcla de distribuciones gaussianas, lo que le permite adaptarse dinámicamente a variaciones graduales de iluminación y sombras. Al colocar un objeto sobre la trampilla, los píxeles de ese objeto ya no corresponden al modelo de fondo, por lo que el algoritmo los etiqueta como 'primer plano' (foreground), generando una máscara binaria.

```

# =====
# DETECCIÓN - MOG2 + contornos (fallback)
# =====
def _detect_mog2(self, frame_bgr, bg_sub, kernel):
    """Devuelve la mejor caja (x1, y1, x2, y2) o None."""
    mask = bg_sub.apply(frame_bgr)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

```

```
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
mask = cv2.dilate(mask, kernel, iterations=2)

contours, _ = cv2.findContours(
    mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)
best_cnt, best_area = None, 0
for cnt in contours:
    a = cv2.contourArea(cnt)
    if a > MIN_AREA and a > best_area:
        best_cnt, best_area = cnt, a

if best_cnt is None:
    return None

x, y, w, h = cv2.boundingRect(best_cnt)
x1 = max(0, x)
y1 = max(0, y)
x2 = min(frame_bgr.shape[1], x + w)
y2 = min(frame_bgr.shape[0], y + h)
return x1, y1, x2, y2
```

7.4 Pipeline de Inferencia del Modelo

Una vez extraído el ROI del objeto, este sigue el siguiente pipeline de preprocesamiento e inferencia antes de tomar una decisión de clasificación:

1. Normalización: los valores de píxel del ROI (rango 0-255, BGR) se convierten al rango [0, 1] dividiendo por 255.0. Además, se aplica la función `preprocess_input()` de MobileNetV2 para centrar los datos según las estadísticas de ImageNet.
2. Expansión de dimensiones: se añade una dimensión de batch usando `np.expand_dims(roi, axis=0)`, transformando el array de forma (160,160,3) a (1,160,160,3), requerido por el modelo.
3. Predicción: `model.predict()` ejecuta la pasada hacia adelante (forward pass) y retorna un array de probabilidades de forma (1,2), donde el índice 0 corresponde a 'Aluminio' y el índice 1 a 'Plástico'.
4. Umbralización: si la probabilidad máxima supera el umbral de confianza configurable (`threshold_slider`, por defecto 0.85), se toma la decisión de clasificación. De lo contrario, el objeto no se clasifica.
5. Acción: la predicción aceptada actualiza la GUI (etiqueta, color, estadísticas) y dispara el comando serial hacia el Arduino.

8. Integración Mecánica y Comunicación Serial

8.1 Protocolo de Comunicación Python → Arduino

La comunicación entre el host (computadora con Python) y el microcontrolador (Arduino Uno) se implementa mediante el protocolo UART (Universal Asynchronous Receiver-Transmitter) a través del puerto USB, utilizando la biblioteca PySerial en Python. El protocolo es deliberadamente simple y robusto: Python envía un único carácter ASCII codificado para minimizar la latencia de comunicación y la complejidad del firmware en el Arduino.

Carácter Enviado	Significado / Acción en el Arduino
'A' (0x41)	Material clasificado como Aluminio. Arduino mueve el servo a la posición 0° (izquierda), abriendo la trampilla hacia el contenedor de aluminio.
'P' (0x50)	Material clasificado como Plástico. Arduino mueve el servo a la posición 90° o 180° (derecha), dirigiendo el residuo al contenedor de plástico.

```
# Fragmento de main.py – Inicialización y Envío Serial con PySerial
import serial
import serial.tools.list_ports

def conectar_serial(self):
    """Establece conexión con el Arduino a través del puerto COM/ttyUSB."""
    try:
        self.ser = serial.Serial(
            port=self.port_var.get(),      # Puerto seleccionado en la GUI
            baudrate=9600,                 # Velocidad de comunicación (bps)
            timeout=1                      # Timeout de lectura en segundos
        )
        self.lbl_estado_serial.config(text='Serial: Conectado', fg='green')
    except serial.SerialException as e:
        messagebox.showerror('Error Serial', str(e))

def enviar_comando(self, clase):
    """Envía el carácter de clasificación al Arduino."""
    if self.ser and self.ser.is_open:
        comando = 'A' if clase == 'Aluminio' else 'P'
        self.ser.write(comando.encode('utf-8')) # Envío como bytes
        import time
        time.sleep(2) # Espera a que el servo complete el movimiento
        self.ser.write(b'C') # Comando 'Close': cierra la trampilla
```


8.2 Firmware del Arduino — Código Fuente Completo

El siguiente código en C++ (Arduino) debe cargarse en el Arduino Uno mediante el Arduino IDE. El firmware espera instrucciones en el puerto serie, controla el servomotor según el comando recibido y devuelve la trampilla a la posición cerrada (neutra) después de un retardo.

```
#include <Servo.h>
#include <Arduino.h>

// =====
// DECLARACIÓN DE SERVOS Y PINES
// =====
Servo servoClasificacion; // Fase 1
Servo servoTrampilla1;    // Fase 2 (Inicia en 180)
Servo servoTrampilla2;    // Fase 2 (Inicia en 0)

int pinClasificacion = 2;
int pinTrampilla1 = 3;
int pinTrampilla2 = 4;

void setup() {
  // 1. INICIAMOS LA COMUNICACIÓN SERIAL (Debe coincidir con Python)
  Serial.begin(9600);

  servoClasificacion.attach(pinClasificacion);
  servoTrampilla1.attach(pinTrampilla1);
  servoTrampilla2.attach(pinTrampilla2);

  // Posiciones iniciales de todos los servos antes de empezar
  servoClasificacion.write(45);
  servoTrampilla1.write(180);
  servoTrampilla2.write(0);

  // Tiempo para que todos se acomoden al inicio
  delay(2000);
}

void loop() {
  // 2. VERIFICAMOS SI PYTHON ENVIÓ UN DATO
  if (Serial.available() > 0) {
    char comando = Serial.read(); // Leemos el carácter recibido

    // Si es Aluminio o Plástico, iniciamos la secuencia
    if (comando == 'A' || comando == 'P') {

      // =====
      // FASE 1: CLASIFICACIÓN
      // =====

      if (comando == 'A') {
        Serial.println("Aluminio detectado!");
        // Movimiento para ALUMINIO (45 a 140)
        for (int ang = 45; ang <= 140; ang++) {
          servoClasificacion.write(ang);
          delay(20);
        }
      }
      else if (comando == 'P') {
        Serial.println("Plástico detectado!");
      }
    }
  }
}
```

```
// Movimiento para PLÁSTICO (Aquí puedes cambiar los ángulos si
// necesitas que gire hacia el lado contrario, por ej: 45 a 0)
// Por ahora mantenemos tu movimiento original:
for (int ang = 45; ang <= 140; ang++) {
    servoClasificacion.write(ang);
    delay(20);
}

// 2 segundos en la posición de clasificación
delay(2000);

// Movimiento de regreso (140 a 45)
for (int ang = 140; ang >= 45; ang--) {
    servoClasificacion.write(ang);
    delay(20);
}
```

9. Guía de Replicabilidad

Esta sección proporciona las instrucciones completas y ordenadas para que cualquier persona pueda replicar el proyecto desde cero en un entorno diferente. Siga cada paso en el orden indicado.

Paso 1 — Preparación del Entorno de Desarrollo

6. Instalar Python 3.9 o superior desde <https://www.python.org/downloads/>. Durante la instalación, marcar la opción 'Add Python to PATH'.
7. Verificar la instalación ejecutando en la terminal: `python --version`
8. (Opcional pero recomendado) Crear un entorno virtual para aislar las dependencias del proyecto:

```
# Crear entorno virtual
python -m venv venv

# Activar el entorno virtual
# En Windows:
venv\Scripts\activate
# En macOS / Linux:
source venv/bin/activate
```

Paso 2 — Instalación de Dependencias Python

Con el entorno virtual activado, ejecute el siguiente comando en la terminal para instalar todas las bibliotecas requeridas:

```
pip install tensorflow==2.12.0 opencv-python pyserial pillow numpy
pip install scikit-learn matplotlib seaborn jupyter

# Verificar instalación de TensorFlow
python -c "import tensorflow as tf; print('TF Version:', tf.__version__)"

# Verificar disponibilidad de GPU (opcional)
python -c "import tensorflow as tf; print('GPUs:',
tf.config.list_physical_devices('GPU'))"
```

Paso 3 — Preparación y Entrenamiento del Modelo

9. Descargar el dataset desde el enlace proporcionado en la Sección 10 y descomprimirlo.
10. Organizar el dataset en la siguiente estructura de directorios:

```
dataset/
├── aluminio/ # Imágenes de aluminio para entrenamiento
└── plastico/ # Imágenes de plástico para entrenamiento
```

11. Abrir el notebook `nvomodelo.ipynb` con Jupyter Notebook o JupyterLab:

```
jupyter notebook nvomodelo.ipynb
```

12. Ajustar la variable DATASET_PATH en el notebook para que apunte al directorio del dataset descargado.
13. Ejecutar todas las celdas del notebook en orden (Kernel → Restart & Run All). Al finalizar, se generará el archivo mobileNetV2v.keras en el directorio de trabajo.

Paso 4 — Configuración y Carga del Firmware Arduino

14. Descargar e instalar el Arduino IDE 2.x desde <https://www.arduino.cc/en/software>
15. Conectar el Arduino Uno a la computadora mediante el cable USB Tipo-A a Tipo-B.
16. Conectar el servomotor SG90 al Arduino de la siguiente manera:
 - Cable Rojo (VCC) → Pin 5V del Arduino
 - Cable Café/Negro (GND) → Pin GND del Arduino
 - Cable Amarillo/Naranja (Señal PWM) → Pin Digital 2,3,4 del Arduino
17. Abrir el Arduino IDE, pegar el código de la Sección 8.2 en un nuevo sketch.
18. Seleccionar la placa correcta: Herramientas → Placa → Arduino Uno
19. Seleccionar el puerto COM correcto: Herramientas → Puerto → COM# (el que aparezca como 'Arduino Uno')
20. Compilar y cargar el sketch presionando el botón 'Subir' (→). Esperar el mensaje 'Subida completada'.
21. Verificar el funcionamiento abriendo el Monitor Serie (Tools → Serial Monitor) a 9600 bps. Debe aparecer el mensaje de inicialización.

Paso 5 — Montaje de la Estructura Física

- Construir la estructura de soporte (caja de madera o acrílico) con dimensiones aproximadas de 110×40×40 cm.
- Instalar la trampilla (lámina de madera delgada o acrílico) con una bisagra en el centro superior, de modo que el servomotor pueda girarla en ambas direcciones.
- Fijar el servomotor en la parte superior de la estructura, alineado con el eje de la bisagra de la trampilla. Conectar el brazo del servo a la trampilla con una varilla rígida.
- Montar la cámara web en una posición cenital (vista desde arriba) apuntando directamente hacia la superficie de la trampilla, a una distancia de aproximadamente 25-35 cm.
- Posicionar dos contenedores (bolsas, cajas) bajo la trampilla, uno a cada lado, para recibir el aluminio y el plástico respectivamente.

Paso 6 — Ejecución de la Aplicación Principal

22. Asegurarse de que el Arduino está conectado y el modelo .keras entrenado existe en el directorio del proyecto.
23. Verificar el nombre del puerto COM del Arduino (en Windows: Administrador de dispositivos → Puertos COM y LPT).
24. Ejecutar la aplicación principal desde la terminal (con el entorno virtual activo):

```
python main.py
```

25. En la interfaz gráfica que se abre:

- Sección 'Modelo': hacer clic en 'Cargar Modelo' y seleccionar el archivo `mobileNetV2v.keras`.
- Sección 'Puerto Serial': seleccionar el puerto COM del Arduino y hacer clic en 'Conectar Serial'.
- Sección 'Cámara': seleccionar el índice de la cámara web (0 para la predeterminada) y hacer clic en 'Iniciar Cámara'.
- Ajustar el control deslizante de 'Umbral de Confianza' según las condiciones de iluminación del entorno (valor recomendado: 0.85).

26. Esperar 3-5 segundos para que el sustractor de fondo MOG2 aprenda el modelo de fondo estático.

27. Colocar un objeto (lata de aluminio o envase de plástico) sobre la trampilla. El sistema detectará, clasificará y accionará el servo automáticamente.

10. Enlaces a Recursos del Proyecto

Recurso	URL de Acceso
Dataset de Imágenes	dataset
Video Demostrativo del Proyecto	VideoPrueba.mp4
Código Fuente	Clasificador
Notebook de Entrenamiento	Clasificador
Dataset TrashNet (referencia)	https://github.com/garythung/trashnet
Documentación MobileNetV2	https://keras.io/api/applications/mobilenet/
Documentación OpenCV MOG2	https://docs.opencv.org/4.x/
Documentación PySerial	https://pyserial.readthedocs.io/en/latest/

11. Anexos

Anexo A — Código Completo del Entrenamiento

```
# %%
# %% [1] Aumento de datos con ImageDataGenerator
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import numpy as np
import matplotlib.pyplot as plt

# AJUSTE 1: Augmentation más suave y realista
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=15,
    zoom_range=[0.8, 1.2],          # Antes 0.5 a 1.5 (muy extremo)
    brightness_range=[0.8, 1.2],   # Antes 0.4 a 1.5 (muy oscuro)
    horizontal_flip=True,
    validation_split=0.2
)

data_gen_entrenamiento = datagen.flow_from_directory(
    "dataset", target_size=(224, 224),
    batch_size=32, shuffle=True, subset="training"
)

data_gen_validacion = datagen.flow_from_directory(
    "dataset", target_size=(224, 224),
    batch_size=32, shuffle=True, subset="validation"
)

# Visualizar muestras
for imagen, etiqueta in data_gen_entrenamiento:
    for i in range(10):
        plt.subplot(2, 5, i+1)
        plt.xticks([]); plt.yticks([])
        plt.imshow(imagen[i])
    break
plt.show()

# %%
# %% [2] Transferencia de aprendizaje (Fine-Tuning)
import tensorflow as tf

mobilenetv2 = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet',
    pooling='avg'
)

# AJUSTE 2: Descongelamos el modelo base para que aprenda TUS materiales
```

```
mobilenetv2.trainable = True

modelo = tf.keras.Sequential([
    mobilenetv2,
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.BatchNormalization(), # Ayuda a estabilizar derivadas
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Dense(2, activation='softmax')
])

# AJUSTE 3: Learning Rate bajo (0.0001) para no olvidar lo que la IA ya sabía
modelo.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

modelo.summary()

# %%
# %% [3] Entrenamiento con Callbacks
import datetime

EPOCHAS = 100

log_dir = "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")

mis_callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        filepath='MobileNetV2v.keras',
        monitor='val_accuracy',
        save_best_only=True,
        verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=15,
        restore_best_weights=True,
        verbose=1
    ),
    tf.keras.callbacks.TensorBoard(log_dir=log_dir, histogram_freq=1)
]

historial = modelo.fit(
    data_gen_entrenamiento,
    epochs=EPOCHAS,
    validation_data=data_gen_validacion,
    callbacks=mis_callbacks
)

# %%
# %% [4] Graficas de precisión y pérdida
acc = historial.history['accuracy']
val_acc = historial.history['val_accuracy']
loss = historial.history['loss']
val_loss = historial.history['val_loss']
range_epocas = range(len(acc))
```



```

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range_epocas, acc, label='Entrenamiento')
plt.plot(range_epocas, val_acc, label='Validación')
plt.title('Precisión')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range_epocas, loss, label='Entrenamiento')
plt.plot(range_epocas, val_loss, label='Validación')
plt.title('Pérdida')
plt.legend()
plt.show()

# %%
# %% [5] Matriz de Confusión y ROC (CORREGIDO)
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, roc_curve, auc
import matplotlib.pyplot as plt
import numpy as np

print("Recalculando métricas correctamente...")

# 1. EL FIX: Redefinimos el generador SIN SHUFFLE para la evaluación
data_gen_evaluacion = datagen.flow_from_directory(
    "dataset",
    target_size=(224, 224),
    batch_size=32,
    shuffle=False, # <--- ¡ESTE ES EL SECRETO!
    subset="validation"
)

# 2. Obtenemos las etiquetas reales (que ahora sí coincidirán)
y_true = data_gen_evaluacion.classes

# 3. Hacemos las predicciones (ahora en orden)
y_pred_proba = modelo.predict(data_gen_evaluacion)
y_pred = np.argmax(y_pred_proba, axis=1)

# --- GRAFICAR MATRIZ ---
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=list(data_gen_evaluacion.class_indices.keys()))
disp.plot(cmap=plt.cm.Blues)
plt.title('Matriz de Confusión')
plt.show()

# --- GRAFICAR ROC ---
fpr, tpr, _ = roc_curve(y_true, y_pred_proba[:, 1])
roc_auc = auc(fpr, tpr)
plt.figure()
plt.plot(fpr, tpr, color='darkorange', label=f'ROC area = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.title('Curva ROC')
plt.legend()
plt.show()

# %%
# %% [6] Herramienta Grad-CAM (Diagnóstico Visual con Etiquetas)
import cv2
import numpy as np

```

```

import tensorflow as tf
import matplotlib.pyplot as plt

def visualizar_gradcam(img_batch, etiqueta_real_batch, model,
last_conv_layer_name='Conv_1'):
    base_model = model.layers[0]

    grad_model = tf.keras.models.Model(
        inputs=base_model.inputs,
        outputs=[base_model.get_layer(last_conv_layer_name).output, base_model.output]
    )

    with tf.GradientTape() as tape:
        conv_outputs, base_model_outputs = grad_model(img_batch)

        x = base_model_outputs
        for layer in model.layers[1:]:
            x = layer(x)

        preds = x
        pred_index = tf.argmax(preds[0]).numpy()
        class_channel = preds[:, pred_index]

        grads = tape.gradient(class_channel, conv_outputs)
        pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

        conv_outputs = conv_outputs[0]
        heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
        heatmap = tf.squeeze(heatmap)

        heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
        heatmap = heatmap.numpy()

        # --- NUEVA LÓGICA DE ETIQUETAS ---
        clases = ['Aluminio', 'Plástico']

        # Extraemos el índice real (convierte de [0,1] a 1, o de [1,0] a 0)
        real_index = np.argmax(etiqueta_real_batch[0])

        nombre_real = clases[real_index]
        nombre_prediccion = clases[pred_index]

        # Validación visual rápida
        icono = "☑" if real_index == pred_index else "✗"

        # --- VISUALIZACIÓN ---
        plt.imshow(img_batch[0])
        heatmap_resized = cv2.resize(heatmap, (224, 224))
        plt.imshow(heatmap_resized, cmap='jet', alpha=0.5)

        # Título profesional con ambas métricas
        plt.title(f'{icono} Real: {nombre_real} | IA dice: {nombre_prediccion}')
        plt.axis('off')
        plt.show()

# Probar Grad-CAM pasándole tanto la imagen como su etiqueta real
img_test, etiqueta_real = next(data_gen_evaluacion)

# Nota: le pasamos img_test[0:1] para mandar solo la primera foto del lote
# y etiqueta_real[0:1] para mandar solo la etiqueta de esa foto

```

```

visualizar_gradcam(img_test[0:1], etiqueta_real[0:1], modelo)

# %%
# %% [7] Clasificación de una imagen externa
from PIL import Image
import requests
from io import BytesIO

url =
"https://media.es.wired.com/photos/66858822d846484dcb12c64b/1:1/w_5473,h_5473,c_limit/2158735448"
def categorizar(url):
    respuesta = requests.get(url)
    img = Image.open(BytesIO(respuesta.content)).convert('RGB')
    img = np.array(img).astype(float) / 255.0
    img = cv2.resize(img, (224, 224))
    prediccion = modelo.predict(img.reshape(1, 224, 224, 3))
    clase = np.argmax(prediccion[0])
    print(f"Predicción: {clase} ({'Plástico' if clase==1 else 'Aluminio'})")
    return clase

categorizar(url)

```

Anexo B — Código Completo de la Interfaz

```

"""
=====
CLASIFICADOR DE RESIDUOS - TRAMPILLA INTELIGENTE
ClasificadorGUI | Tkinter + OpenCV + TensorFlow + PySerial
Clases detectadas: "Aluminio" y "Plástico"
=====
Dependencias:
    pip install opencv-python tensorflow pyserial numpy Pillow
=====
"""

# — Suprimir mensajes de TensorFlow antes de cualquier importación —
import os
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0"
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import threading
import time
import queue
import cv2
import numpy as np
from PIL import Image, ImageTk

# — PySerial (opcional) —
try:
    import serial
    import serial.tools.list_ports
    SERIAL_OK = True
except ImportError:
    SERIAL_OK = False

```

```

# --- TensorFlow (opcional) -----
try:
    import tensorflow as tf
    TF_OK = True
except ImportError:
    TF_OK = False

# =====
#  CONSTANTES
#  =====
CLASS_NAMES      = ["Aluminio", "Plástico"]
INPUT_SIZE       = (224, 224)
COOLDOWN_SEG     = 2.5      # segundos entre detecciones confirmadas
MIN_AREA         = 3000     # área mínima de contorno (px²)
HITS_CONFIRM     = 5        # frames consecutivos para confirmar objeto
FRAME_BUF        = 2        # tamaño de cola de frames

CLASS_COLORS = {
    "Aluminio":    (0, 200, 255),
    "Plástico":    (0, 255, 80),
    "Desconocido": (160, 160, 160),
}

# =====
#  PALETA DE COLORES DE LA GUI
#  =====
BG          = "#0d1117"
PANEL_BG   = "#161b22"
BORDER      = "#30363d"
ACCENT      = "#58a6ff"
GREEN       = "#3fb950"
RED         = "#f85149"
YELLOW      = "#d29922"
DARK_BTN    = "#21262d"
TEXT_MAIN   = "#e6edf3"
TEXT_DIM    = "#8b949e"

FONT_TITLE  = ("Consolas", 11, "bold")
FONT_LABEL  = ("Consolas", 9)
FONT_VALUE  = ("Consolas", 22, "bold")
FONT_BTN    = ("Consolas", 9, "bold")
FONT_SMALL  = ("Consolas", 8)

# =====
#  HILO DE VISIÓN E INFERENCIA
#  =====
class VisionThread(threading.Thread):
    """
    Captura frames, detecta objetos y realiza inferencia. Corre como daemon.

    Modos de detección (en orden de prioridad):
    1. SSDLite (.tflite) – detector de cajas, luego clasificador Keras sobre ROI
    2. MOG2 + contornos – fallback si no hay modelo SSD cargado
    """

    def __init__(self, cam_index, model_path, ssd_path,
                  frame_q, event_q, conf_var, stop_evt):
        super().__init__(daemon=True)

```

```

        self.cam_index = cam_index
        self.model_path = model_path # .keras / .h5 → clasificador
        self.ssd_path = ssd_path # .tflite → detector (opcional)
        self.frame_q = frame_q
        self.event_q = event_q
        self.conf_var = conf_var
        self.stop_evt = stop_evt

        self.model = None # modelo Keras
        self.ssd = None # intérprete TFLite
        self.ssd_input = None # detalle del tensor de entrada SSD
        self.ssd_size = (300, 300) # resolución de entrada SSD por defecto

# =====
# CARGA DE MODELOS
# =====
def _load_classifier(self):
    """Carga el clasificador Keras (.keras / .h5)."""
    if not TF_OK or not self.model_path:
        return
    try:
        gpus = tf.config.list_physical_devices("GPU")
        if gpus:
            tf.config.experimental.set_memory_growth(gpus[0], True)
            self.model = tf.keras.models.load_model(self.model_path)
            print(f"[OK] Clasificador: {os.path.basename(self.model_path)}")
    except Exception as exc:
        print(f"[ERROR] Clasificador: {exc}")

def _load_ssd(self):
    """
    Carga el detector SSDLite (.tflite).
    Detecta automáticamente el tamaño de entrada y el tipo de dato.
    """
    if not TF_OK or not self.ssd_path:
        return
    try:
        interp = tf.lite.Interpreter(model_path=self.ssd_path)
        interp.allocate_tensors()

        inp = interp.get_input_details()[0]
        h = inp["shape"][1]
        w = inp["shape"][2]
        self.ssd_size = (w, h)
        self.ssd_input = inp
        self.ssd = interp
        print(f"[OK] SSD detector: {os.path.basename(self.ssd_path)}"
              f" entrada={w}x{h} dtype={inp['dtype'].__name__}")
    except Exception as exc:
        print(f"[ERROR] SSD: {exc}")

# =====
# DETECCIÓN - SSDLite TFLite
# =====
def _detect_ssd(self, frame_bgr):
    """
    Corre inferencia con el modelo SSDLite.
    Devuelve la mejor caja (x1, y1, x2, y2) en coordenadas de píxel,
    o None si no hay detección por encima del umbral mínimo SSD (0.3).
    """

```

```

Formato de salida estándar TF OD API (4 tensores):
[0] boxes      → shape (1, N, 4)  [ymin, xmin, ymax, xmax] normalizados
[1] classes    → shape (1, N)
[2] scores     → shape (1, N)
[3] num_dets   → shape (1,)
"""
h_f, w_f = frame_bgr.shape[:2]
w_in, h_in = self.ssd_size

# Preprocesar al tamaño y dtype del modelo
img = cv2.resize(frame_bgr, (w_in, h_in))
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

dtype = self.ssd_input["dtype"]
if dtype == np.uint8:
    tensor = img.astype(np.uint8)
else:
    tensor = (img / 255.0).astype(np.float32)
tensor = np.expand_dims(tensor, axis=0)

self.ssd.set_tensor(self.ssd_input["index"], tensor)
self.ssd.invoke()

out_details = self.ssd.get_output_details()

# Ordenar salidas por índice para seguir la convención TF OD API
out_details_sorted = sorted(out_details, key=lambda d: d["index"])

if len(out_details_sorted) < 3:
    return None # formato desconocido

boxes = self.ssd.get_tensor(out_details_sorted[0]["index"])[0]
scores = self.ssd.get_tensor(out_details_sorted[2]["index"])[0]

# Filtrar: score mínimo fijo en 0.3 (independiente del slider)
SSD_MIN_SCORE = 0.30
best_score = -1.0
best_box = None

for i, score in enumerate(scores):
    if score > SSD_MIN_SCORE and score > best_score:
        best_score = score
        best_box = boxes[i] # [ymin, xmin, ymax, xmax] normalizado

if best_box is None:
    return None

ymin, xmin, ymax, xmax = best_box
x1 = max(0, int(xmin * w_f))
y1 = max(0, int(ymin * h_f))
x2 = min(w_f, int(xmax * w_f))
y2 = min(h_f, int(ymax * h_f))

# Rechazar cajas demasiado pequeñas
if (x2 - x1) * (y2 - y1) < MIN_AREA:
    return None

return x1, y1, x2, y2

# =====

```

```

# DETECCIÓN - MOG2 + contornos (fallback)
# =====
def _detect_mog2(self, frame_bgr, bg_sub, kernel):
    """Devuelve la mejor caja (x1, y1, x2, y2) o None."""
    mask = bg_sub.apply(frame_bgr)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    mask = cv2.dilate(mask, kernel, iterations=2)

    contours, _ = cv2.findContours(
        mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )
    best_cnt, best_area = None, 0
    for cnt in contours:
        a = cv2.contourArea(cnt)
        if a > MIN_AREA and a > best_area:
            best_cnt, best_area = cnt, a

    if best_cnt is None:
        return None

    x, y, w, h = cv2.boundingRect(best_cnt)
    x1 = max(0, x)
    y1 = max(0, y)
    x2 = min(frame_bgr.shape[1], x + w)
    y2 = min(frame_bgr.shape[0], y + h)
    return x1, y1, x2, y2

# =====
# CLASIFICACIÓN - Keras
# =====
def _preprocess(self, roi_bgr):
    img = cv2.resize(roi_bgr, INPUT_SIZE)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = img.astype(np.float32) / 255.0
    return np.expand_dims(img, axis=0)

def _classify(self, roi_bgr):
    """Devuelve (etiqueta, confianza) o ('Desconocido', 0)."""
    if self.model is None:
        return "Desconocido", 0.0
    preds = self.model.predict(self._preprocess(roi_bgr), verbose=0)[0]
    if len(preds) == 1:
        # sigmoid
        conf = float(preds[0])
        return (CLASS_NAMES[1], conf) if conf >= 0.5 \
            else (CLASS_NAMES[0], 1.0 - conf)
    else:
        # softmax
        idx = int(np.argmax(preds))
        conf = float(preds[idx])
        lbl = CLASS_NAMES[idx] if idx < len(CLASS_NAMES) else "Desconocido"
        return lbl, conf

# =====
# DIBUJO
# =====
def _draw_box(self, frame, x1, y1, x2, y2, label, conf, mode_tag):
    color = CLASS_COLORS.get(label, (160, 160, 160))
    tag_text = f"{{label}}  {{conf*100:.1f}}%  [{{mode_tag}}]"
    cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
    (tw, th), _ = cv2.getTextSize(tag_text, cv2.FONT_HERSHEY_SIMPLEX, 0.60, 2)

```

```
        cv2.rectangle(frame, (x1, y1 - th - 10), (x1 + tw + 8, y1), color, -1)
        cv2.putText(frame, tag_text, (x1 + 4, y1 - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.60, (20, 20, 20), 2)

# =====
# BUCLE PRINCIPAL
# =====
def run(self):
    self._load_classifier()
    self._load_ssd()

    cap = cv2.VideoCapture(self.cam_index)
    if not cap.isOpened():
        self.event_q.put(("error", "No se pudo abrir la cámara."))
        return

    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

    # Fallback MOG2 (siempre se inicializa, solo se usa si no hay SSD)
    bg_sub = cv2.createBackgroundSubtractorMOG2(
        history=300, varThreshold=40, detectShadows=False
    )
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))

    # Notificar modo activo
    mode = "SSD" if self.ssd is not None else "MOG2"
    self.event_q.put(("info", f"Detector activo: {mode}"))

    last_time = 0.0
    last_label = None
    consec_hits = 0

    while not self.stop_evt.is_set():
        ret, frame = cap.read()
        if not ret:
            time.sleep(0.04)
            continue

        display = frame.copy()
        threshold = self.conf_var.get() / 100.0
        now = time.time()

        # --- Detección ---
        if self.ssd is not None:
            box = self._detect_ssd(frame)
            mode_tag = "SSD"
        else:
            box = self._detect_mog2(frame, bg_sub, kernel)
            mode_tag = "MOG2"

        det_label, det_conf = None, 0.0

        if box is not None:
            x1, y1, x2, y2 = box
            roi = frame[y1:y2, x1:x2]

            label, conf = ("Desconocido", 0.0)
            if roi.size > 0:
                label, conf = self._classify(roi)
```



```

        self._draw_box(display, x1, y1, x2, y2, label, conf, mode_tag)

        if conf >= threshold and label != "Desconocido":
            det_label, det_conf = label, conf

        # --- Cooldown + confirmación ---
        if det_label:
            if det_label == last_label:
                consec_hits += 1
            else:
                consec_hits, last_label = 1, det_label

            if consec_hits >= HITS_CONFIRM and now - last_time >= COOLDOWN_SEG:
                last_time = now
                consec_hits = 0
                self.event_q.put(("detection", det_label, det_conf))
        else:
            consec_hits = 0

        # --- Overlay de estado ---
        cd_rest = max(0.0, COOLDOWN_SEG - (now - last_time))
        cv2.putText(
            display,
            f"Umbral:{self.conf_var.get():.0f}%  CD:{cd_rest:.1f}s
[{mode_tag}]",
            (8, 22), cv2.FONT_HERSHEY_SIMPLEX, 0.48, (210, 210, 210), 1
        )

        # --- Enviar frame a GUI ---
        try:
            self.frame_q.put_nowait(display)
        except queue.Full:
            pass

    cap.release()

# =====
# CLASE PRINCIPAL DE LA GUI
# =====
class ClasificadorGUI:
    """Interfaz gráfica principal del clasificador de residuos."""

    def __init__(self, root: tk.Tk):
        self.root = root
        self.root.title("Clasificador de Residuos IA")
        self.root.configure(bg=BG)
        self.root.resizable(True, True)
        self.root.minsize(960, 640)

        # --- Variables de estado ---
        self.model_path = tk.StringVar(value="(ningún modelo seleccionado)")
        self.ssd_path = tk.StringVar(value="(opcional - sin SSD)")
        self.cam_index = tk.IntVar(value=0)
        self.port_var = tk.StringVar(value="")
        self.conf_var = tk.DoubleVar(value=70.0)
        self.count_alu = tk.IntVar(value=0)
        self.count_plas = tk.IntVar(value=0)

```

```

self.serial_conn = None
self.vision_thread = None
self.stop_evt = threading.Event()
self.frame_q = queue.Queue(maxsize=FRAME_BUF)
self.event_q = queue.Queue()
self._cam_running = False

# — Construir UI por secciones —————
self._apply_combobox_style()
self._build_layout()
self._build_video_panel()
self._build_side_panel()

self._refresh_ports()
self._poll() # arrancar ciclo de actualización

# =====
# ESTILOS
# =====
def _apply_combobox_style(self):
    style = ttk.Style()
    style.theme_use("default")
    style.configure("TCombobox",
        fieldbackground=DARK_BTN, background=DARK_BTN,
        foreground=TEXT_MAIN, selectbackground=DARK_BTN,
        selectforeground=TEXT_MAIN, bordercolor=BORDER,
        arrowcolor=ACCENT)

# =====
# LAYOUT PRINCIPAL (grid: col 0 = video expandible, col 1 = panel fijo)
# =====
def _build_layout(self):
    self._main = tk.Frame(self.root, bg=BG)
    self._main.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
    self._main.columnconfigure(0, weight=1) # video se estira
    self._main.columnconfigure(1, weight=0) # panel lateral fijo
    self._main.rowconfigure(0, weight=1)

# =====
# PANEL DE VIDEO (col 0)
# =====
def _build_video_panel(self):
    vf = tk.Frame(self._main, bg=PANEL_BG,
        highlightbackground=BORDER, highlightthickness=1)
    vf.grid(row=0, column=0, sticky="nsew", padx=(0, 6))
    vf.rowconfigure(2, weight=1)
    vf.columnconfigure(0, weight=1)

    tk.Label(vf, text="● FEED EN VIVO",
        bg=PANEL_BG, fg=ACCENT, font=FONT_TITLE,
        anchor="w", padx=10).grid(row=0, column=0,
            sticky="ew", pady=(6, 2))
    tk.Frame(vf, bg=BORDER, height=1).grid(row=1, column=0, sticky="ew")

    self.canvas = tk.Label(
        vf, bg="#0a0a0a",
        text="Cámara no iniciada\n\nSelecciona modelo → Iniciar Cámara",
        fg=TEXT_DIM, font=FONT_LABEL, justify=tk.CENTER
    )
    self.canvas.grid(row=2, column=0, sticky="nsew", padx=6, pady=6)

```

```

# =====
# PANEL LATERAL (col 1, ancho fijo 300 px)
# =====
def _build_side_panel(self):
    # — Contenedor exterior (fijo 300 px)
    outer = tk.Frame(self._main, bg=PANEL_BG, width=300,
                     highlightbackground=BORDER, highlightthickness=1)
    outer.grid(row=0, column=1, sticky="nsew")
    outer.pack_propagate(False)
    outer.rowconfigure(0, weight=1)
    outer.columnconfigure(0, weight=1)

    # — Canvas desplazable
    self._side_canvas = tk.Canvas(
        outer, bg=PANEL_BG, highlightthickness=0, bd=0
    )
    self._side_canvas.grid(row=0, column=0, sticky="nsew")

    # — Barra de desplazamiento
    vsb = tk.Scrollbar(
        outer, orient=tk.VERTICAL,
        command=self._side_canvas.yview,
        bg=DARK_BTN, troughcolor=PANEL_BG,
        activebackground=ACCENT, relief=tk.FLAT
    )
    vsb.grid(row=0, column=1, sticky="ns")
    self._side_canvas.configure(yscrollcommand=vsb.set)

    # — Frame interior (contenido real)
    self._side = tk.Frame(self._side_canvas, bg=PANEL_BG)
    self._side_window = self._side_canvas.create_window(
        (0, 0), window=self._side, anchor="nw"
    )

    # Actualizar región desplazable cuando cambie el tamaño interior
    self._side.bind("<Configure>", self._on_side_configure)
    # Ajustar ancho del frame interior al canvas
    self._side_canvas.bind("<Configure>", self._on_canvas_configure)

    # Scroll con rueda del ratón
    self._side_canvas.bind_all("<MouseWheel>", self._on_mousewheel)

    self._build_section_model()
    self._build_section_camera()
    self._build_section_serial()
    self._build_section_threshold()
    self._build_section_stats()
    self._build_section_log()

    # — Callbacks de scroll
    def _on_side_configure(self, event):
        """Actualiza la región desplazable cuando el contenido cambia de tamaño."""
        self._side_canvas.configure(
            scrollregion=self._side_canvas.bbox("all")
        )

    def _on_canvas_configure(self, event):
        """Mantiene el frame interior con el mismo ancho que el canvas."""
        self._side_canvas.itemconfig(self._side_window, width=event.width)

```

```

def _on_mousewheel(self, event):
    """Desplaza el panel lateral con la rueda del ratón."""
    self._side_canvas.yview_scroll(int(-1 * (event.delta / 120)), "units")

# — Helpers reutilizables —
def _section_title(self, text):
    """Título de sección con separador debajo."""
    tk.Label(self._side, text=text,
              bg=PANEL_BG, fg=ACCENT, font=FONT_TITLE,
              anchor="w", padx=10
              ).pack(fill=tk.X, pady=(10, 2))
    tk.Frame(self._side, bg=BORDER, height=1).pack(fill=tk.X, padx=10)

def _row(self):
    """Frame contenedor para una fila del panel."""
    f = tk.Frame(self._side, bg=PANEL_BG)
    f.pack(fill=tk.X, padx=10, pady=3)
    return f

def _btn(self, parent, text, cmd, color=ACCENT):
    return tk.Button(
        parent, text=text, command=cmd,
        bg=DARK_BTN, fg=color, font=FONT_BTN,
        activebackground="#30363d", activeforeground=color,
        relief=tk.FLAT, cursor="hand2", padx=8, pady=4,
        highlightthickness=1, highlightbackground=BORDER
    )

# =====
# SECCIÓN 1 – MODELO
# =====
def _build_section_model(self):
    self._section_title("① MODELOS DE IA")

    # — Clasificador Keras —
    tk.Label(self._side, text="Clasificador (.keras / .h5)",
              bg=PANEL_BG, fg=TEXT_DIM, font=FONT_SMALL,
              anchor="w", padx=12).pack(fill=tk.X, pady=(4, 0))

    r = self._row()
    self._btn(r, " Buscar clasificador", self._select_model, ACCENT
              ).pack(fill=tk.X)

    self._lbl_model = tk.Label(
        self._side, textvariable=self.model_path,
        bg=PANEL_BG, fg=TEXT_DIM, font=FONT_SMALL,
        wraplength=265, anchor="w", padx=12
    )
    self._lbl_model.pack(fill=tk.X, pady=(0, 2))

    # — Separador interno —
    tk.Frame(self._side, bg=BORDER, height=1
              ).pack(fill=tk.X, padx=10, pady=(4, 0))

    # — Detector SSDLite —
    r_ssd = tk.Frame(self._side, bg=PANEL_BG)
    r_ssd.pack(fill=tk.X, padx=12, pady=(4, 0))

    tk.Label(r_ssd, text="Detector SSDLite (.tflite)",

```

```

        bg=PANEL_BG, fg=TEXT_DIM, font=FONT_SMALL,
        anchor="w").pack(side=tk.LEFT)

    # Badge que indica si SSD está cargado
    self._lbl_ssd_badge = tk.Label(
        r_ssd, text="OFF", bg="#2d1f0e", fg=TEXT_DIM,
        font=("Consolas", 7, "bold"), padx=4
    )
    self._lbl_ssd_badge.pack(side=tk.RIGHT)

    r = self._row()
    self._btn(r, "  Buscar modelo SSD", self._select_ssd, YELLOW
        ).pack(side=tk.LEFT, fill=tk.X, expand=True)
    self._btn(r, "X", self._clear_ssd, RED
        ).pack(side=tk.LEFT, padx=(4, 0))

    self._lbl_ssd = tk.Label(
        self._side, textvariable=self.ssd_path,
        bg=PANEL_BG, fg=TEXT_DIM, font=FONT_SMALL,
        wraplength=265, anchor="w", padx=12
    )
    self._lbl_ssd.pack(fill=tk.X, pady=(0, 2))

    # Nota informativa
    tk.Label(
        self._side,
        text="Sin SSD → usa MOG2 (sustracción de fondo)",
        bg=PANEL_BG, fg="#444d56", font=("Consolas", 7),
        anchor="w", padx=12
    ).pack(fill=tk.X, pady=(0, 4))

    # =====
    # SECCIÓN 2 - CÁMARA
    # =====
    def _build_section_camera(self):
        self._section_title("②  CÁMARA")

        r = self._row()
        tk.Label(r, text="Índice:", bg=PANEL_BG, fg=TEXT_DIM,
            font=FONT_LABEL, width=8, anchor="w").pack(side=tk.LEFT)
        ttk.Combobox(r, textvariable=self.cam_index,
            values=[0, 1, 2, 3], width=5,
            state="readonly").pack(side=tk.LEFT, padx=4)

        r = self._row()
        self._btn_cam = self._btn(r, "▶  Iniciar Cámara",
            self._toggle_camera, GREEN)
        self._btn_cam.pack(fill=tk.X)

    # =====
    # SECCIÓN 3 - SERIAL
    # =====
    def _build_section_serial(self):
        self._section_title("③  ARDUINO - SERIAL")

        r = self._row()
        tk.Label(r, text="Puerto:", bg=PANEL_BG, fg=TEXT_DIM,
            font=FONT_LABEL, width=8, anchor="w").pack(side=tk.LEFT)
        self._port_cb = ttk.Combobox(r, textvariable=self.port_var,
            values=[], width=11, state="readonly")

```

```

self._port_cb.pack(side=tk.LEFT, padx=4)
self._btn(r, "🔄", self._refresh_ports, TEXT_DIM
            ).pack(side=tk.LEFT, padx=2)

r = self._row()
self._btn_ser = self._btn(r, "⚡ Conectar Arduino",
                           self._toggle_serial, YELLOW)
self._btn_ser.pack(fill=tk.X)

r = self._row()
self._lbl_ser = tk.Label(r, text="● Desconectado",
                          bg=PANEL_BG, fg=RED, font=FONT_SMALL,
                          anchor="w")
self._lbl_ser.pack(fill=tk.X)

# =====
# SECCIÓN 4 - UMBRAL DE CONFIANZA
# =====
def _build_section_threshold(self):
    self._section_title("④ UMBRAL DE CONFIANZA")

    r = self._row()
    self._lbl_conf = tk.Label(r, text="70 %",
                              bg=PANEL_BG, fg=ACCENT,
                              font=("Consolas", 16, "bold"), width=6)
    self._lbl_conf.pack(side=tk.RIGHT)

    self._scale = tk.Scale(
        self._side, from_=0, to=100,
        orient=tk.HORIZONTAL, variable=self.conf_var,
        command=self._on_conf_change,
        bg=PANEL_BG, fg=TEXT_MAIN, troughcolor=DARK_BTN,
        highlightthickness=0, sliderrelief=tk.FLAT,
        activebackground=ACCENT, font=FONT_SMALL,
        showvalue=False, length=240
    )
    self._scale.pack(fill=tk.X, padx=10)

    r = self._row()
    tk.Label(r, text="0 %", bg=PANEL_BG, fg=TEXT_DIM,
             font=FONT_SMALL).pack(side=tk.LEFT)
    tk.Label(r, text="100 %", bg=PANEL_BG, fg=TEXT_DIM,
             font=FONT_SMALL).pack(side=tk.RIGHT)

# =====
# SECCIÓN 5 - ESTADÍSTICAS
# =====
def _build_section_stats(self):
    self._section_title("⑤ ESTADÍSTICAS")

    sf = tk.Frame(self._side, bg=PANEL_BG)
    sf.pack(fill=tk.X, padx=10, pady=6)
    sf.columnconfigure(0, weight=1)
    sf.columnconfigure(1, weight=1)

    # Tarjeta Aluminio
    ca = tk.Frame(sf, bg="#0d1f2d",
                  highlightbackground="#00c8ff", highlightthickness=1)
    ca.grid(row=0, column=0, padx=(0, 4), sticky="nsew")
    tk.Label(ca, text="Aluminio", bg="#0d1f2d",

```

```

        fg="#00c8ff", font=FONT_SMALL).pack(pady=(6, 0))
    tk.Label(ca, textvariable=self.count_alu, bg="#0d1f2d",
            fg="#00c8ff", font=FONT_VALUE).pack()
    tk.Label(ca, text="detectados", bg="#0d1f2d",
            fg=TEXT_DIM, font=FONT_SMALL).pack(pady=(0, 6))

    # Tarjeta Plástico
    cp = tk.Frame(sf, bg="#0d2d1a",
            highlightbackground=GREEN, highlightthickness=1)
    cp.grid(row=0, column=1, padx=(4, 0), sticky="nsew")
    tk.Label(cp, text="Plástico", bg="#0d2d1a",
            fg=GREEN, font=FONT_SMALL).pack(pady=(6, 0))
    tk.Label(cp, textvariable=self.count_plas, bg="#0d2d1a",
            fg=GREEN, font=FONT_VALUE).pack()
    tk.Label(cp, text="detectados", bg="#0d2d1a",
            fg=TEXT_DIM, font=FONT_SMALL).pack(pady=(0, 6))

    r = self._row()
    self._btn(r, "⌛  Resetear Contadores",
            self._reset_counters, RED).pack(fill=tk.X)

    # =====
    # SECCIÓN 6 – LOG
    # =====
    def _build_section_log(self):
        self._section_title("⑥  LOG DE EVENTOS")

        lf = tk.Frame(self._side, bg=PANEL_BG)
        lf.pack(fill=tk.BOTH, expand=True, padx=10, pady=(4, 10))

        sb = tk.Scrollbar(lf, bg=PANEL_BG, troughcolor=PANEL_BG)
        sb.pack(side=tk.RIGHT, fill=tk.Y)

        self._log_box = tk.Text(
            lf, bg="#0a0e14", fg=TEXT_DIM, font=FONT_SMALL,
            height=7, state=tk.DISABLED, relief=tk.FLAT, wrap=tk.WORD,
            highlightbackground=BORDER, highlightthickness=1,
            yscrollcommand=sb.set
        )
        self._log_box.pack(fill=tk.BOTH, expand=True)
        sb.config(command=self._log_box.yview)

        self._log_box.tag_configure("alu", foreground="#00c8ff")
        self._log_box.tag_configure("plas", foreground=GREEN)
        self._log_box.tag_configure("err", foreground=RED)
        self._log_box.tag_configure("serial", foreground=YELLOW)
        self._log_box.tag_configure("info", foreground=TEXT_DIM)

        self._log("Sistema iniciado.", "info")

    # =====
    # LÓGICA – MODELO
    # =====
    def _select_model(self):
        path = filedialog.askopenfilename(
            title="Seleccionar clasificador",
            filetypes=[("Modelos Keras", "*.keras *.h5"),
                      ("Todos", "*.*")]
        )
        if path:

```

```
        self.model_path.set(path)
        self._log(f"Clasificador: {os.path.basename(path)}", "info")

    def _select_ssd(self):
        path = filedialog.askopenfilename(
            title="Seleccionar detector SSDLite (.tflite)",
            filetypes=[("TFLite", "*.tflite"),
                       ("Todos", "*.*")]
        )
        if path:
            self.ssd_path.set(path)
            self._lbl_ssd_badge.configure(
                text=" SSD ", bg="#1f3d1f", fg=GREEN
            )
            self._log(f"SSD detector: {os.path.basename(path)}", "info")

    def _clear_ssd(self):
        self.ssd_path.set("(opcional - sin SSD)")
        self._lbl_ssd_badge.configure(text="OFF", bg="#2d1f0e", fg=TEXT_DIM)
        self._log("SSD detector removido → se usará MOG2.", "info")

# =====
# LÓGICA - CÁMARA
# =====
    def _toggle_camera(self):
        if self._cam_running:
            self._stop_camera()
        else:
            self._start_camera()

    def _start_camera(self):
        mp = self.model_path.get()
        if mp == "(ningún modelo seleccionado)":
            if not messagebox.askyesno(
                "Sin modelo",
                "No se ha seleccionado un modelo .keras/.h5.\n"
                "¿Continuar solo con visualización?"
            ):
                return
        mp = ""

        self.stop_evt.clear()
        self._clear_queues()

        # Resolver rutas (vaciar si son los valores por defecto)
        mp = mp if mp else ""
        ssd = self.ssd_path.get()
        ssd = ssd if (ssd and not ssd.startswith("(")) else ""

        self.vision_thread = VisionThread(
            cam_index = self.cam_index.get(),
            model_path = mp,
            ssd_path = ssd,
            frame_q = self.frame_q,
            event_q = self.event_q,
            conf_var = self.conf_var,
            stop_evt = self.stop_evt
        )
        self.vision_thread.start()
        self._cam_running = True
```



```

        self.btn_cam.configure(text="■ Detener Cámara", fg=RED)
        self._log(f"Cámara {self.cam_index.get()} iniciada.", "info")

    def _stop_camera(self):
        self.stop_evt.set()
        self._cam_running = False
        self.btn_cam.configure(text="▶ Iniciar Cámara", fg=GREEN)
        self.canvas.configure(image="",
                                text="Cámara detenida.", fg=TEXT_DIM)
        self._log("Cámara detenida.", "info")

    def _clear_queues(self):
        for q in (self.frame_q, self.event_q):
            while not q.empty():
                try: q.get_nowait()
                except queue.Empty: break

    # =====
    # LÓGICA - SERIAL
    # =====
    def _refresh_ports(self):
        if not SERIAL_OK:
            self.port_cb.configure(values=["(pyserial no instalado)"])
            return
        ports = [p.device for p in serial.tools.list_ports.comports()]
        self.port_cb.configure(values=ports or ["(sin puertos)"])
    if ports:
        self.port_var.set(ports[0])
        self._log(f"Puertos: {ports or 'ninguno'}", "info")

    def _toggle_serial(self):
        if self.serial_conn and self.serial_conn.is_open:
            self._disconnect_serial()
        else:
            self._connect_serial()

    def _connect_serial(self):
        if not SERIAL_OK:
            messagebox.showerror("Error", "pyserial no instalado.")
            return
        port = self.port_var.get()
        if not port or port.startswith("("):
            messagebox.showwarning("Advertencia",
                                    "Selecciona un puerto COM válido.")
            return
        try:
            self.serial_conn = serial.Serial(port, baudrate=9600, timeout=1)
            self._lbl_ser.configure(text=f"● Conectado {port}", fg=GREEN)
            self._btn_ser.configure(text="✕ Desconectar Arduino")
            self._log(f"Serial: {port} @ 9600", "serial")
        except Exception as exc:
            messagebox.showerror("Error Serial", str(exc))
            self._log(f"Error serial: {exc}", "err")

    def _disconnect_serial(self):
        if self.serial_conn:
            try: self.serial_conn.close()
            except Exception: pass
        self.serial_conn = None
        self._lbl_ser.configure(text="● Desconectado", fg=RED)

```

```

self._btn_ser.configure(text="⚡ Conectar Arduino")
self._log("Serial desconectado.", "serial")

def _send_serial(self, char: str):
    if self.serial_conn and self.serial_conn.is_open:
        try:
            self.serial_conn.write(char.encode("ascii"))
            self._log(f"TX → '{char}'", "serial")
        except Exception as exc:
            self._log(f"Error TX: {exc}", "err")

# =====
# LÓGICA - UMBRAL Y CONTADORES
# =====
def _on_conf_change(self, value):
    self._lbl_conf.configure(text=f"{float(value):.0f} %")

def _reset_counters(self):
    self.count_alu.set(0)
    self.count_plas.set(0)
    self._log("Contadores reseteados.", "info")

# =====
# LOG HELPER
# =====
def _log(self, msg: str, tag: str = "info"):
    ts = time.strftime("%H:%M:%S")
    self._log_box.configure(state=tk.NORMAL)
    self._log_box.insert(tk.END, f"[{ts}] {msg}\n", tag)
    self._log_box.see(tk.END)
    self._log_box.configure(state=tk.DISABLED)

# =====
# CICLO DE POLLING (hilo principal, no bloqueante)
# =====
def _poll(self):
    self._poll_frames()
    self._poll_events()
    self.root.after(30, self._poll)

def _poll_frames(self):
    """Toma el frame más reciente de la cola y actualiza el canvas."""
    latest = None
    try:
        while True:
            latest = self.frame_q.get_nowait()
    except queue.Empty:
        pass

    if latest is None:
        return

    cw = self.canvas.winfo_width()
    ch = self.canvas.winfo_height()
    if cw > 10 and ch > 10:
        latest = cv2.resize(latest, (cw, ch),
                           interpolation=cv2.INTER_LINEAR)

    img = Image.fromarray(cv2.cvtColor(latest, cv2.COLOR_BGR2RGB))
    photo = ImageTk.PhotoImage(image=img)

```

```

        self.canvas.configure(image=photo, text="")
        self.canvas._ref = photo          # evitar garbage collection

def _poll_events(self):
    """Procesa detecciones confirmadas y errores del hilo de visión."""
    try:
        while True:
            ev = self.event_q.get_nowait()

            if ev[0] == "detection":
                _, label, conf = ev
                if label == "Aluminio":
                    self.count_alu.set(self.count_alu.get() + 1)
                    self._send_serial("A")
                    self._log(
                        f"✓ ALUMINIO {conf*100:.1f}% "
                        f"[total: {self.count_alu.get()}]", "alu")
                elif label == "Plástico":
                    self.count_plas.set(self.count_plas.get() + 1)
                    self._send_serial("P")
                    self._log(
                        f"✓ PLÁSTICO {conf*100:.1f}% "
                        f"[total: {self.count_plas.get()}]", "plas")

            elif ev[0] == "info":
                self._log(ev[1], "info")

            elif ev[0] == "error":
                messagebox.showerror("Error de cámara", ev[1])
                self._log(f"Error: {ev[1]}", "err")
                self._stop_camera()

        except queue.Empty:
            pass

    # =====
    # CIERRE LIMPIO
    # =====
    def on_close(self):
        self.stop_evt.set()
        self._disconnect_serial()
        if self.vision_thread and self.vision_thread.is_alive():
            self.vision_thread.join(timeout=2.0)
        self.root.destroy()

    # =====
    # PUNTO DE ENTRADA
    # =====
    if __name__ == "__main__":
        root = tk.Tk()
        app = ClasificadorGUI(root)
        root.protocol("WM_DELETE_WINDOW", app.on_close)

        # Centrar en pantalla
        W, H = 1100, 700
        root.update_idletasks()
        sx = (root.winfo_screenwidth() - W) // 2
        sy = (root.winfo_screenheight() - H) // 2
        root.geometry(f"{W}x{H}+{sx}+{sy}")

```

```
root.mainloop()
```

Anexo C — Código Completo del Firmware Arduino

El código completo del firmware para el Arduino Uno se encuentra documentado íntegramente en la Sección 8.2 del presente manual. Se recomienda guardar el archivo con el nombre `main.cpp` para mantener la convención de nomenclatura del Arduino IDE.

```
#include <Servo.h>
#include <Arduino.h>

// =====
// DECLARACIÓN DE SERVOS Y PINES
// =====
Servo servoClasificacion; // Fase 1
Servo servoTrampilla1;    // Fase 2 (Inicia en 180)
Servo servoTrampilla2;    // Fase 2 (Inicia en 0)

int pinClasificacion = 2;
int pinTrampilla1 = 3;
int pinTrampilla2 = 4;

void setup() {
  // 1. INICIAMOS LA COMUNICACIÓN SERIAL (Debe coincidir con Python)
  Serial.begin(9600);

  servoClasificacion.attach(pinClasificacion);
  servoTrampilla1.attach(pinTrampilla1);
  servoTrampilla2.attach(pinTrampilla2);

  // Posiciones iniciales de todos los servos antes de empezar
  servoClasificacion.write(45);
  servoTrampilla1.write(180);
  servoTrampilla2.write(0);

  // Tiempo para que todos se acomoden al inicio
  delay(2000);
}

void loop() {
  // 2. VERIFICAMOS SI PYTHON ENVIÓ UN DATO
  if (Serial.available() > 0) {
    char comando = Serial.read(); // Leemos el carácter recibido

    // Si es Aluminio o Plástico, iniciamos la secuencia
    if (comando == 'A' || comando == 'P') {

      // =====
      // FASE 1: CLASIFICACIÓN
      // =====

      if (comando == 'A') {
        Serial.println("Aluminio detectado!");
        // Movimiento para ALUMINIO (45 a 140)
        for (int ang = 45; ang <= 140; ang++) {
          servoClasificacion.write(ang);
          delay(20);
        }
      }
    }
  }
}
```

```
    }
    else if (comando == 'P') {
        Serial.println("Plástico detectado!");
        // Movimiento para PLÁSTICO (Aquí puedes cambiar los ángulos si
        // necesitas que gire hacia el lado contrario, por ej: 45 a 0)
        // Por ahora mantenemos tu movimiento original:
        for (int ang = 45; ang <= 140; ang++) {
            servoClasificacion.write(ang);
            delay(20);
        }
    }

    // 2 segundos en la posición de clasificación
    delay(2000);

    // Movimiento de regreso (140 a 45)
    for (int ang = 140; ang >= 45; ang--) {
        servoClasificacion.write(ang);
        delay(20);
    }

    // 2 segundos de espera antes de activar las trampillas
    delay(2000);

    // =====
    // FASE 2: ACTIVACIÓN DE TRAMPILLAS (SIMULTÁNEO)
    // =====

    // Movimiento de apertura sincronizado
    for (int i = 0; i <= 45; i++) {
        servoTrampilla1.write(180 - i); // Baja de 180 a 135
        servoTrampilla2.write(0 + i);   // Sube de 0 a 45
        delay(20); // Movimiento suave
    }

    // 2 segundos de espera con las trampillas abiertas para que caiga la pieza
    delay(2000);

    // Movimiento de cierre sincronizado
    for (int i = 45; i >= 0; i--) {
        servoTrampilla1.write(180 - i); // Sube de 135 a 180
        servoTrampilla2.write(0 + i);   // Baja de 45 a 0
        delay(20);
    }

    // 2 segundos de espera con las trampillas cerradas antes de reiniciar
    delay(2000);
}
}
```